Chapter 24

Customizing and Extending Rails

As you've come to learn, Rails provides an answer for almost every question you have about building a modern web application. It provides the basics for handling requests, accessing a database, writing user interfaces, and running tests. It does this by having a tightly integrated design, which is often referred to as Rails being "opinionated software."

This tight coupling comes at a price. If, for example, the way Rails manages CSS doesn't meet the needs of your project, you could be in trouble. Or if you prefer to write your tests in a different way, Rails doesn't give you a lot of options. Or does it? In the early days of Rails, customizing it was difficult or impossible. Starting with Rails 3, much effort was expended to make Rails more customizable. With Rails 7, developers have the flexibility to use the tools they prefer or that work the way they want to work. That's what we'll explore in this chapter.

We'll replace four parts of Rails in this chapter. First, we'll write a Web Component instead of using Stimulus. Then we'll see how to use RSpec instead of Rails' default testing library to write our tests. Next, we'll replace ERB for the alternative templating language Slim. Finally, we'll see how to manage CSS using Webpack instead of putting it in `app/assets/stylesheets`. This chapter will demonstrate another benefit to Rails, which is that you don't have to throw out the parts that work for you to use alternatives that work better. Let's get started.